Majordome

Walter Dal'Maz Silva

2026-03-14

Table of contents

1 Majordome Home **4**
1.1 Getting Started . . . 4

2 Energy sources **5**
2.1 CombustionAtmosphereCHON . . . 5
2.2 CombustionPowerSupply . . . 5
2.3 Energy sources . . . 6
2.3.1 HeatedGasEnergySource . . . 6
2.3.2 CombustionEnergySource . . . 7

3 Reactor models **10**
3.1 Warnings . . . 10
3.2 Utilities . . . 10
3.3 NormalFlowRate . . . 14
3.4 Plug-flow reactors . . . 16
3.4.1 PlugFlowAxialSources . . . 16
3.4.2 PlugFlowChainCantera . . . 17

4 Symbolic functions **20**
4.1 PiecewiseFunction . . . 20
4.2 Symbolic species parameterization . . . 22
4.2.1 Nasa7Thermo . . . 24
4.3 Symbolic mixture properties . . . 27

5 Transport properties **28**
5.1 EffectiveThermalConductivity . . . 28
5.2 SolutionDimless . . . 30
5.3 SutherlandFitting . . . 31
5.4 WSGGRadlibBordbar2020 . . . 32
5.4.1 Validation of Bordbar WSGG implementation . . . 33
5.4.2 Application to radiative combustion . . . 33

6 Fluent files **35**
6.1 FluentFvParticlesParser . . . 35
6.2 FluentInterpolationParser . . . 35

7 Common **37**
7.1 RelaxUpdate and StabilizeNvarsConvergenceCheck . . . 37

8 Documents **39**
8.1 Tutorial . . . 40
8.2 Examples . . . 42

9 Argument parsing **46**
9.1 FuncArguments . . . 46

9.2 Creating class constructors . . . 47

10 PDF Tools **49**
10.1 PdfToTextConverter . . . 49

11 Plotting **51**
11.1 MajordomePlot . . . 51
11.2 PowerFormatter . . . 56
11.3 General utilities . . . 57

12 Developer Guide **59**
12.1 Prerequisites . . . 59
12.2 Command reference . . . 59
12.2.1 Windows PowerShell . . . 59
12.2.2 Unix-like shell . . . 60
12.3 Documentation . . . 60
12.4 Distribution . . . 61
12.5 Dependency management . . . 61
12.6 Todo lists . . . 61
12.6.1 Quick needs . . . 61
12.6.2 Wishlist . . . 62

13 References **63**

1 Majordome Home

1.1 Getting Started

To register `majordome` as a Jupyter kernel, run:

```
majordome --install-kernel
```

You can customize the kernel identity if needed:

```
majordome --install-kernel `
    --kernel-name majordome-dev `
    --display-name "Majordome (dev)"
```

2 Energy sources

aka majordome_engineering.energy

This module is concerned with providing classes for computation of energy sources; these are mostly related to estimation of fuel consumption or integration to another simulation modules. Hereafter we illustrate the in cascading complexity the available energy sources. Some of these are simple wrapper around features provided by the combustion utilities.

2.1 CombustionAtmosphereCHON

A common need in industry is to evaluate the real heating value from a gas composition reported by the supplier. This is the main goal of `CombustionAtmosphereCHON`, which has a very simple API provided below:

```
gas = CombustionAtmosphereCHON("gri30.yaml")
lhv = gas.solution_heating_value("CH4: 1", "O2: 1")
print(f"Solution heating value: {lhv:.2f} MJ/kg")
```

```
Solution heating value: 50.03 MJ/kg
```

2.2 CombustionPowerSupply

Built upon `CombustionAtmosphereCHON`, `CombustionPowerSupply` performs the basic calculations for retrieving required flow rates from a given power specification; this proves very handy for the CFD engineer.

```
supply = CombustionPowerSupply(500.0, 1.0, "CH4: 1", "O2: 1", "gri30.yaml")
print(supply.report())
```

```
| Property                   | Unit    |    Value |
|----------------------------|---------|----------|
| Required power             | kW      | 500      |
| Lower heating value        | MJ/kg   |  50.0254 |
| Fuel mass flow rate        | kg/h    |  35.9817 |
| Oxidizer mass flow rate    | kg/h    | 143.532  |
| Total mass flow rate       | kg/h    | 179.514  |
| Fuel volume flow rate      | Nm³/h   |  50.2707 |
| Oxidizer volume flow rate  | Nm³/h   | 100.541  |
| Total volume flow rate     | Nm³/h   | 150.812  |
| Water production           | kg/h    |  80.8092 |
| Carbon dioxide production  | kg/h    |  98.7047 |
| Total emissions            | kg/h    | 179.514  |
```

2.3 Energy sources

2.3.1 HeatedGasEnergySource

```
source = HeatedGasEnergySource("airish.yaml", 500.0, mass_flow_rate=1.0)
print(source.report())
```

```
| Property                | Unit      | Value                  |
|-------------------------|----------|-----------------------|
| Source kind             |          | HeatedGasEnergySource |
| Provided power          | kW       | 500.0                 |
| Source                  |          | airish.yaml           |
| Phase                   |          | air                   |
| Reference area          | m²       | 1.0                   |
| Mass flow rate          | kg/s     | 1.0                   |
| Volume flow rate        | m³/s     | 2.2051590808893957    |
| Momentum flux           | kg.m/s²  | 2.2051590808893957    |
| Reference temperature   | K        | 298.15                |
| Reference pressure      | Pa       | 101325.0              |
| Temperature             | K        | 778.5219246537463     |
| Pressure                | Pa       | 101324.99999999999    |
| Density                 | kg/m³    | 0.453482022529039     |
| Specific enthalpy       | J/(kg.K) | 500038.49937969144    |
| Specific heat capacity  | J/(kg.K) | 1091.748083028256     |
| mass: AR                | -        | 0.013790127718329308  |
| mass: N2                | -        | 0.7542602692440457    |
| mass: O2                | -        | 0.23194960303762516   |
```

```
source = HeatedGasEnergySource("airish.yaml", 500.0, mass_flow_rate=1.0,
                               cross_area=0.1, Y="N2: 0.79, O2: 0.21")
print(source.report())
```

```
| Property                | Unit      | Value                  |
|-------------------------|----------|-----------------------|
| Source kind             |          | HeatedGasEnergySource |
| Provided power          | kW       | 500.0                 |
| Source                  |          | airish.yaml           |
| Phase                   |          | air                   |
| Reference area          | m²       | 0.1                   |
| Mass flow rate          | kg/s     | 1.0                   |
| Volume flow rate        | m³/s     | 2.2090693490835482    |
| Momentum flux           | kg.m/s²  | 22.090693490835484    |
| Reference temperature   | K        | 298.15                |
| Reference pressure      | Pa       | 101325.0              |
| Temperature             | K        | 774.4142430033143     |
| Pressure                | Pa       | 101324.99999999999    |
| Density                 | kg/m³    | 0.45267931512193527   |
| Specific enthalpy       | J/(kg.K) | 500040.3236104384     |
| Specific heat capacity  | J/(kg.K) | 1100.3346580471573    |
| mass: N2                | -        | 0.79                  |
| mass: O2                | -        | 0.21                  |
```

2.3.2 CombustionEnergySource

```
fuel_state = StateType("CH4: 1",             300, 101325)
oxid_state = StateType("N2: 0.79, O2: 0.21", 300, 101325)

ops = CombustionPowerOp(500.0, 1.0, fuel_state, oxid_state)
source = CombustionEnergySource("ch4/bfer.yaml", operation=ops, cross_area=0.1)

# Recover values for other tests that follow
# TODO wrap these in properties for clean API:
mdot_fuel = source._qty_fuel.mass
mdot_oxid = source._qty_oxid.mass

nfr_fuel = NormalFlowRate.new_from_solution(source._qty_fuel)
nfr_oxid = NormalFlowRate.new_from_solution(source._qty_oxid)

qdot_fuel = 3600 * mdot_fuel / nfr_fuel.density
qdot_oxid = 3600 * mdot_oxid / nfr_oxid.density

print(source.report())
```

```
| Property                | Unit      | Value                   |
|-------------------------|-----------|-------------------------|
| Source kind             |           | CombustionEnergySource  |
| Provided power          | kW        | 500.0                   |
| Source                  |           | ch4/bfer.yaml           |
| Phase                   |           | CH4_BFER                |
| Reference area          | m²        | 0.1                     |
| Mass flow rate          | kg/s      | 0.18117768922524702     |
| Volume flow rate        | m³/s      | 1.2210278725486656      |
| Momentum flux           | kg.m/s²   | 2.2122300842798666      |
| ******* FLUE:           |           |                         |
| Temperature             | K         | 2257.7969133307734      |
| Pressure                | Pa        | 101325.00000819027      |
| Density                 | kg/m³     | 0.14838128866548536     |
| Specific enthalpy       | J/(kg.K)  | -254492.89776989678     |
| Specific heat capacity  | J/(kg.K)  | 1516.597922218735       |
| mass: CH4               | -         | 2.7662274427222582e-17  |
| mass: CO                | -         | 0.010559409120413024    |
| mass: CO2               | -         | 0.13474113875913193     |
| mass: H2O               | -         | 0.12389490081229192     |
| mass: N2                | -         | 0.7247731344386522      |
| mass: O2                | -         | 0.006031416869510841    |
| ******* FUEL:           |           |                         |
| Temperature             | K         | 300.0                   |
| Pressure                | Pa        | 101325.00000000001      |
| Density                 | kg/m³     | 0.6516985521312658      |
| Specific enthalpy       | J/(kg.K)  | -4645856.881890704      |
| Specific heat capacity  | J/(kg.K)  | 2229.0429122783953      |
| mass: CH4               | -         | 1.0                     |
| ******* OXIDIZER:       |           |                         |
```

```
| Temperature            | K        | 300.0                  |
| Pressure               | Pa       | 101325.00000000001     |
| Density                | kg/m³    | 1.171970349439655      |
| Specific enthalpy      | J/(kg.K) | 1907.6015935135354     |
| Specific heat capacity | J/(kg.K) | 1010.0686132988887     |
| mass: N2               | -        | 0.7670907820415769     |
| mass: O2               | -        | 0.2329092179584231     |
```

```
ops = CombustionFlowOp("mass", mdot_fuel, mdot_oxid, fuel_state, oxid_state)
source = CombustionEnergySource("ch4/bfer.yaml", operation=ops, cross_area=0.1)
print(source.report())
```

```
| Property               | Unit     | Value                  |
|------------------------|----------|------------------------|
| Source kind            |          | CombustionEnergySource |
| Provided power         | kW       | 500.0                  |
| Source                 |          | ch4/bfer.yaml          |
| Phase                  |          | CH4_BFER               |
| Reference area         | m²       | 0.1                    |
| Mass flow rate         | kg/s     | 0.18117768922524702    |
| Volume flow rate       | m³/s     | 1.2210278725486656     |
| Momentum flux          | kg.m/s²  | 2.2122300842798666     |
| ******* FLUE:          |          |                        |
| Temperature            | K        | 2257.7969133307734     |
| Pressure               | Pa       | 101325.00000819027     |
| Density                | kg/m³    | 0.14838128866548536    |
| Specific enthalpy      | J/(kg.K) | -254492.89776989678    |
| Specific heat capacity | J/(kg.K) | 1516.597922218735      |
| mass: CH4              | -        | 2.7662274427222582e-17 |
| mass: CO               | -        | 0.010559409120413024   |
| mass: CO2              | -        | 0.13474113875913193    |
| mass: H2O              | -        | 0.12389490081229192    |
| mass: N2               | -        | 0.7247731344386522     |
| mass: O2               | -        | 0.006031416869510841   |
| ******* FUEL:          |          |                        |
| Temperature            | K        | 300.0                  |
| Pressure               | Pa       | 101325.00000000001     |
| Density                | kg/m³    | 0.6516985521312658     |
| Specific enthalpy      | J/(kg.K) | -4645856.881890704     |
| Specific heat capacity | J/(kg.K) | 2229.0429122783953     |
| mass: CH4              | -        | 1.0                    |
| ******* OXIDIZER:      |          |                        |
| Temperature            | K        | 300.0                  |
| Pressure               | Pa       | 101325.00000000001     |
| Density                | kg/m³    | 1.171970349439655      |
| Specific enthalpy      | J/(kg.K) | 1907.6015935135354     |
| Specific heat capacity | J/(kg.K) | 1010.0686132988887     |
| mass: N2               | -        | 0.7670907820415769     |
| mass: O2               | -        | 0.2329092179584231     |
```

```
ops = CombustionFlowOp("volume", qdot_fuel, qdot_oxid, fuel_state, oxid_state)
source = CombustionEnergySource("ch4/bfer.yaml", operation=ops, cross_area=0.1)
print(source.report())
```

Property	Unit	Value
Source kind		CombustionEnergySource
Provided power	kW	500.0
Source		ch4/bfer.yaml
Phase		CH4_BFER
Reference area	m^2	0.1
Mass flow rate	kg/s	0.18117768922524705
Volume flow rate	m^3/s	1.2210278725486658
Momentum flux	$kg.m/s^2$	2.212230084279867
******* FLUE:		
Temperature	K	2257.7969133307734
Pressure	Pa	101325.00000819027
Density	kg/m^3	0.14838128866548536
Specific enthalpy	J/(kg.K)	-254492.89776989678
Specific heat capacity	J/(kg.K)	1516.597922218735
mass: CH4	-	2.7662274427222582e-17
mass: CO	-	0.010559409120413024
mass: CO2	-	0.13474113875913193
mass: H2O	-	0.12389490081229192
mass: N2	-	0.7247731344386522
mass: O2	-	0.006031416869510841
******* FUEL:		
Temperature	K	300.0
Pressure	Pa	101325.00000000001
Density	kg/m^3	0.6516985521312658
Specific enthalpy	J/(kg.K)	-4645856.881890704
Specific heat capacity	J/(kg.K)	2229.0429122783953
mass: CH4	-	1.0
******* OXIDIZER:		
Temperature	K	300.0
Pressure	Pa	101325.00000000001
Density	kg/m^3	1.171970349439655
Specific enthalpy	J/(kg.K)	1907.6015935135354
Specific heat capacity	J/(kg.K)	1010.0686132988887
mass: N2	-	0.7670907820415769
mass: O2	-	0.2329092179584231

3 Reactor models

aka majordome_engineering.reactor

3.1 Warnings

Module warnings can be controlled throught the following attributes:

toggle_reactor_warnings

```
toggle_reactor_warnings(**kwargs : Any): -> None:
```

Reverse truth value of warning flags.

Parameters

```
toggle_non_key_value : bool | None = None
```

If true, reverse truth value of warning about non key-value composition.

```
toggle_missing_species_name : bool | None = None
```

If true, reverse truth value of warning about missing species name.

```
toggle_unknown_species : bool | None = None
```

If true, reverse truth value of warning about unknown species.

3.2 Utilities

Let's start by creating a standard Cantera solution:

```
solution = ct.Solution("airish.yaml")
solution.TPY = 273.15, ct.one_atm, "N2: 0.78 O2: 0.21, AR: 0.01"
```

Conversion of composition strings with name filtering is available:

composition_to_dict

```
composition_to_dict(
    Y : str,
    species_names : list[str] = []
    ) -> dict[str, float]:
```

Convert a Cantera composition string to dictionary.

Parameters

`Y : str`

Composition specification string, *e.g.* "O2: 1, N2: 3".

`species_names : list[str] = []`

List of valid species names for validation. If provided, only species in this list will be included in the output dictionary. If not provided, all species will be included.

```
composition_to_dict("O2: 1, teste: 1", solution.species_names)
```

```
{'O2': 1.0}
```

It is also possible to set unit composition with species names; if you do not provide a validation list, all are are kept:

```
composition_to_dict("O2, N2, hello")
```

```
{'O2': 1.0, 'N2': 1.0, 'hello': 1.0}
```

When working with arrays, take care not to end up in the following situation:

```
composition_to_array(", teste: 1", solution.species_names)
```

```
array([0., 0., 0.])
```

There is also a helper for generating inputs for use with `tabulate.tabulate`:

solution_report

```
solution_report(
    sol : cantera.composite.Solution | cantera.composite.Quantity,
    specific_props : bool = True,
    composition_spec : str = 'mass',
    selected_species : list[str] = [],
    **kwargs : Any
```

```
    ) -> list[tuple[str, str, typing.Any]]:
```

Generate a solution report for tabulation.

Parameters

`sol : cantera.composite.Solution | cantera.composite.Quantity`

Cantera solution object for report generation.

`specific_props : bool = True`

If true, add specific heat capacity and enthalpy.

`composition_spec : str = 'mass'`

Composition units specification, `mass` or `mole`.

`selected_species : list[str] = []`

Selected species to display; return all if a composition specification was provided.

`show_mass : bool  = 'False'`

If true, add mass of quantity to report if `sol` is a `ct.composite.Quantity`.

```
data = solution_report(solution, specific_props=True,
                       composition_spec="mass", selected_species=[])
print(tabulate(data))
```

```
----------------------  --------  ------------
Temperature             K            273.15
Pressure                Pa        101325
Density                 kg/m³          1.28735
Specific enthalpy       J/(kg.K)  -25114.9
Specific heat capacity  J/(kg.K)    1004.95
mass: AR                -              0.01
mass: N2                -              0.78
mass: O2                -              0.21
----------------------  --------  ------------
```

Because sometimes Cantera lacks hard-copy utilities for certain classes, we provide simple wrappers that create new instances and set the state to the same of the source object. Nothing checked, nothing tested. A first of this kind is `copy_solution`, illustrated below:

copy_solution

`copy_solution(sol : Solution): -> Solution:`

Makes a hard copy of a Solution object.

Parameters

`sol : Solution`

Solution to be copied.

```
newairs = copy_solution(solution)
newairs.TPX = 373.15, None, newairs.X
print(tabulate(solution_report(newairs)))
```

```
----------------------  --------  -------------
Temperature             K            373.15
Pressure                Pa        101325
Density                 kg/m³          0.942356
Specific enthalpy       J/(kg.K)   75902.1
Specific heat capacity  J/(kg.K)    1015.82
mass: AR                -              0.01
mass: N2                -              0.78
mass: O2                -              0.21
----------------------  --------  -------------
```

```
print(tabulate(solution_report(solution)))
```

```
----------------------  --------  ------------
Temperature             K            273.15
Pressure                Pa        101325
Density                 kg/m³          1.28735
Specific enthalpy       J/(kg.K)  -25114.9
Specific heat capacity  J/(kg.K)    1004.95
mass: AR                -              0.01
mass: N2                -              0.78
mass: O2                -              0.21
----------------------  --------  ------------
```

In addition to this, there is `copy_quantity`, which proves quite useful in establishing an *algebra* of mixtures.

copy_quantity

`copy_quantity(qty : Quantity): -> Quantity:`

Makes a hard copy of a ct.composite.Quantity object.

Parameters

`qty : Quantity`

Quantity to be copied.

```
air = copy_solution(solution)

air1 = ct.Quantity(air, mass=1.0)
air1.TPX = 373.15, None, newairs.X

air2 = copy_quantity(air1)
air2.TPX = 273.15, None, air2.X

mixair = air1 + air2
air1.T, air2.T, mixair.T
```

```
(373.15, 273.15, 323.337485365699)
```

3.3 NormalFlowRate

Common daily work activity for the process engineer is to perform mass balances, but wait, ..., gas flow rates are generally provided under normal conditions, and compositions may vary, so you need to compute normal densities first... whatever. This class provides a calculator wrapping a Cantera solution object so that your life gets easier.

NormalFlowRate

```
NormalFlowRate(
    mech : str | pathlib._local.Path,
    *,
    X : str | dict[str, float] | None = None,
    Y : str | dict[str, float] | None = None,
    T_ref : float = 273.15,
    P_ref : float = 101325.0,
    name : str | None = None
    ) -> None:
```

Compute normal flow rate for a given composition.
This class makes use of the user defined state to create a function object that converts industrial scale flow rates in normal cubic meters per hour to kilograms per second. Nothing more, nothing less, it aims at helping the process engineer in daily life for this quite repetitive need when performing mass balances.

Parameters

`mech : str | pathlib._local.Path`

Path to Cantera mechanism used to compute mixture properties.

```
X : str | dict[str, float] | None = None
```

Composition specification in mole fractions. Notice that both `X` and `Y` are mutally exclusive keyword arguments.

```
Y : str | dict[str, float] | None = None
```

Composition specification in mass fractions. Notice that both `X` and `Y` are mutally exclusive keyword arguments.

```
T_ref : float = 273.15
```

Reference temperature of the system. If your industry does not use the same standard as the default values, and only in that case, please consider updating this keyword.

```
P_ref : float = 101325.0
```

Reference pressure of the system. If your industry does not use the same standard as the default values, and only in that case, please consider updating this keyword.

```
name : str | None = None
```

Name of phase in mechanism, if more than one are specified within the same Cantera YAML database file.

Its simples use case is as follows:

```
nfr = NormalFlowRate("airish.yaml")
print(f"Convert 1000 Nm³/h to {nfr(1000.0):.5f} kg/s")
```

```
Convert 1000 Nm³/h to 0.35903 kg/s
```

If the database file default composition does not suit you, no problems:

```
nfr = NormalFlowRate("airish.yaml", X="N2: 1")
print(f"Convert 1000 Nm³/h to {nfr(1000.0):.5f} kg/s")
```

```
Convert 1000 Nm³/h to 0.34718 kg/s
```

You can also print a nice report to inspect the details of internal state. For more, please check its API documentation.

```
print(nfr.report())
```

```
|------------------------|----------|--------------|
| Temperature            | K        |    273.15    |
| Pressure               | Pa       | 101325       |
| Density                | kg/m³    |      1.24985 |
| Specific enthalpy      | J/(kg.K) | -25864.9     |
| Specific heat capacity | J/(kg.K) |   1035.52    |
| mass: N2               | -        |      1       |
```

3.4 Plug-flow reactors

3.4.1 PlugFlowAxialSources

PlugFlowAxialSources

```
PlugFlowAxialSources(
    n_reactors : int,
    n_species : int
    ) -> None:
```

Provides a data structure for use with `PlugFlowChainCantera`.
Helper data class for use with the solution method `loop` of the plug-flow reactor implementation.
It provides the required memory for storage of source terms distributed along the reactor.

Parameters

`n_reactors : int`

Number of reactors in chain.

`n_species : int`

Number of species in mechanism.

`Q : NDArray[np.float64] = None`

Array of external heat source [W].

`m : NDArray[np.float64] = None`

Array of axial mass source terms [kg/s].

`h : NDArray[np.float64] = None`

Array of enthalpy of axial mass source terms [J/kg].

`Y : NDArray[np.float64, np.float64] = None`

Array of mass fractions of axial mass source terms [-].

3.4.2 PlugFlowChainCantera

PlugFlowChainCantera

```
PlugFlowChainCantera(
    mechanism : str,
    phase : str,
    z : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]],
    V : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]],
    P : float = 101325.0,
    K : float = 1.0,
```

```
    smoot_flux : bool = False,
    cantera_steady : bool = True
    ) -> None:
```

Plug-flow reactor as a chain of 0-D reactors with Cantera.

Parameters

`mechanism : str`

Name or path to Cantera mechanism to be used.

`phase : str`

Name of phase to simulate (not inferred, even if a single is present!).

`z : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]]`

Spatial coordinates of reactor cells [m].

`V : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]]`

Volumes of reactor cells [m^3].

`P : float = 101325.0`

Reactor operating pressure [Pa].

`K : float = 1.0`

Valve response constant (do not use unless simulation fails).

`smoot_flux : bool = False`

Apply a smoot transition function when internal stepping is performed; this is intended to avoid unphysical steady state approximations.

`cantera_steady : bool = True`

If true, use Cantera' s `advance_to_steady_state` to solve problem; otherwise advance over meaninful time-scale of the problem.

get_reactor_data

```
get_reactor_data(pfr : PlugFlowChainCantera): -> PlugFlowAxialSources:
```

Wrapper to allocate properly dimensioned solver data.

Parameters

`pfr : PlugFlowChainCantera`

Reactor for which data is to be allocated.

4 Symbolic functions

aka majordome_engineering.symbolic

This module contains utilities for working with symbolic functions, especially in the context of engineering problems treated by Majordome models. It is mostly a wrapper around CasADi, organizing symbolic operations and providing additional functionality for function manipulation in a physical context.

4.1 PiecewiseFunction

A major issue when working with symbolic expressions and real-world data is that they do not support flow-control constructs like `if` statements. This is a problem when we want to represent functions that are defined piecewise, especially when algorithmic differentiation is involved.

As an example, take the parameterization of thermodynamic properties of chemical species. For quantitative purposes, to ensure high accuracy, researchers have found that it is best to use different polynomial fits for different temperature ranges. This means that the thermodynamic properties of a species are defined as piecewise functions of temperature, which is a problem when we want to use them in a symbolic context.

To tackle this issue in *certain* contexts, `PiecewiseSymbolicFunction` provides an interpolation between different ranges of the domain of a function. Internally it makes use of Heaviside functions to ensure that the function is continuous. This is not a perfect solution, as it does not guarantee differentiability at the breakpoints, but it is a good compromise between accuracy and simplicity. It is also worth noting that this class is not meant to be used in a general context, but rather in specific contexts where the function is known to be well-behaved.

PiecewiseSymbolicFunction

```
PiecewiseSymbolicFunction(
    breakpoints : list[float],
    functions : list[typing.Any]
    ) -> None:
```

Compose a symbolic piecewise function with CasADi.

Parameters

`breakpoints : list[float]`

List of breakpoints where the function changes.

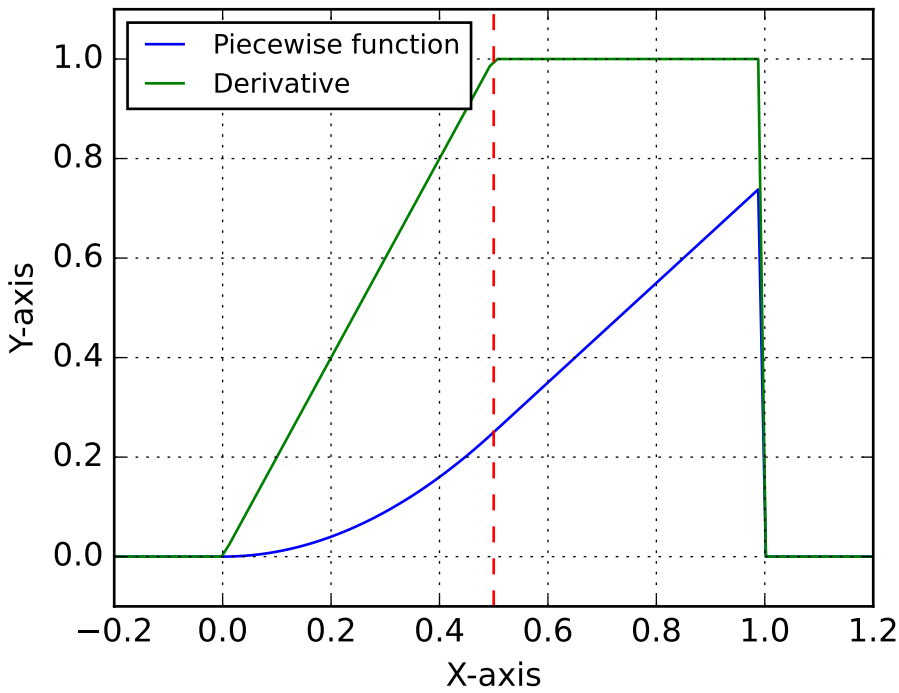
`functions : list[typing.Any]`

> List of functions to apply between breakpoints. The number of functions must be one less than the number of breakpoints.

Let's illustrate its use with a simple example. We define two functions `f` and `g`, and we want to create a piecewise function that is equal to `f` for `x < 0.5` and equal to `g` for `x >= 0.5`. Functions have being created such that they evaluate to the same value at the breakpoint, as the practical case that led to the class development, *i.e.* NASA-parameterization of thermochemical properties. Below we show how to create the piecewise function, evaluate it numerically and symbolically, and compute its derivative symbolically.

```
# Define two simple functions:
f = lambda x: x**2
g = lambda x: f(0.5) + (x - 0.5)

# Create the piecewise function:
F = PiecewiseSymbolicFunction([0, 0.5, 1], [f, g])

# Evaluate numerically:
x = np.linspace(-0.2, 1.2, 100)
y = F(x)

# Evaluate symbolically:
T = SX.sym("T")
Y = F(T)

# Create a function for evaluating the derivative:
ydot = Function("fdot", [T], [jacobian(Y, T)])
```

Below we plot the piecewise function and its derivative. The function is continuous, but the derivative is not, as expected, although the transition is handled smoothly by the Heaviside functions. The red dashed line indicates the breakpoint at `x = 0.5`, where the function transitions from `f` to `g`.



4.2 Symbolic species parameterization

from_species

```
from_species(
    species : Species,
    T : SX
    ) -> AbstractSymbolicThermo:
```

Create a `Nasa7Thermo` object from a Cantera species.

Parameters

`species : Species`

Cantera species object, with NASA7 thermodynamic data.

`T : SX`

Temperature variable (symbolic).

Suppose you need to create a symbolic representation of the NASA7 thermodynamic parameterization of molecular nitrogen, species of index 47 in the GRI-Mech 3.0 mechanism shipped with Cantera. Let's start by loading and retrieving the thermodynamic data for this species.

```
gas = ct.Solution("gri30.yaml")
species = gas.species()[47]
thermo = species.thermo
```

Below we instantiate `SymbolicThermo` with the input data of the species. It is important to emphasize here that when building more complex systems, generally one should share the symbolic variables, *e.g.* temperature, across the different components of the system. For this reason, the constructor of symbolic thermodynamic classes expect the temperature symbol to be provided, instead of trying to create it internally; this is a design choice that will improve code robustness when handling mixtures and avoid many of the pitfalls of symbolic programming.

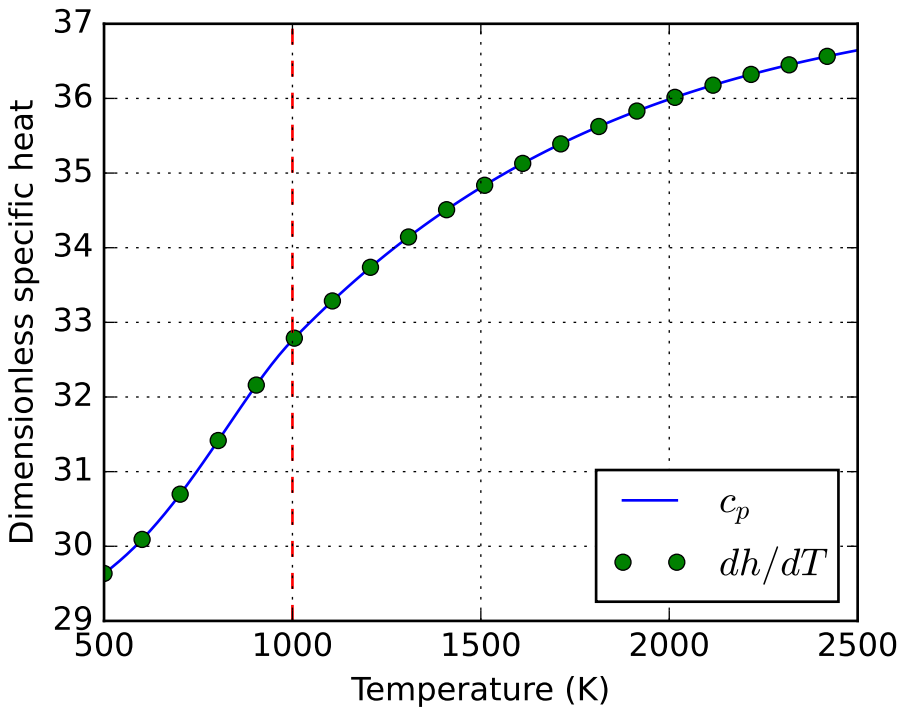
```
T = SX.sym("T")
nasa7 = SymbolicThermo.from_species(species, T)
```

> **Practical usage**
>
> You could instead directly instantiate `Nasa7Thermo` with the input data, but using the factory method `SymbolicThermo.from_species` is a better choice, as it will allow you to easily switch to a different thermodynamic model. Furthermore, it is the safe choice for loading entire databases.
>
> ```
> nasa7 = Nasa7Thermo(T, species.thermo.input_data)
>
> # This is just syntactic sugar for the above:
> nasa7 = Nasa7Thermo.from_species(species, T)
> ```

Here we illustrate the use of algorithmic differentiation to compute the derivative of the specific enthalpy with respect to temperature.

```
hdot = Function("hdot", [T], [jacobian(nasa7.h(T), T)])
```

The above derivative is then confronted to the specific heat below:



Finally, we compare the deviations between symbolic and numerical (Cantera) evaluation of the thermodynamic functions. This is not for error analysis, but a proof of correctness of the symbolic implementation.

```
# XXX: Cantera evaluates quantities per kmol! Divide by 1000 to get
# per mol, as the symbolic implementation does.
c_ct = np.array([0.001 * species.thermo.cp(T) for T in T_num])
h_ct = np.array([0.001 * species.thermo.h(T) for T in T_num])
s_ct = np.array([0.001 * species.thermo.s(T) for T in T_num])

error_c = np.mean(np.abs(nasa7._cp(T_num) - c_ct))
error_h = np.mean(np.abs(nasa7._h(T_num) - h_ct))
error_s = np.mean(np.abs(nasa7._s(T_num) - s_ct))

error_c, error_h, error_s
```

```
(np.float64(4.334310688136611e-15),
 np.float64(6.311893230304122e-12),
 np.float64(3.2400748750660566e-14))
```

4.2.1 Nasa7Thermo

Nasa7Thermo

```
Nasa7Thermo(
    T : SX,
    input_data : dict[str, typing.Any]
    ) -> None:
```

NASA7 thermodynamic parameterization.
This class does not implement Horner polynomial evaluation, as the main use case is to create symbolic expressions that are then evaluated by CasADi, which can handle polynomial evaluation efficiently. This is intentional and allows for easy verification. It aims at providing a similar interface as `SpeciesThermo` from Cantera, from which it retrieves the data. That means, molar properties are provided through `cp`, `h`, and `s` properties.

Parameters

`T : SX`

Temperature variable (symbolic).

`input_data : dict[str, typing.Any]`

NASA7 thermodynamic data, as provided by Cantera's `SpeciesThermo` property `input_data`.

specific_heat

```
specific_heat(
    T : Any,
    a : list[float],
    symbolic : bool = False
    ) -> casadi.casadi.Function | casadi.casadi.SX:
```

Compose NASA7 specific heat parameterization.

Parameters

`T : Any`

Temperature variable (symbolic or numeric).

`a : list[float]`

List of 7 NASA7 coefficients.

`symbolic : bool = False`

Whether to return a symbolic expression.

enthalpy

```
enthalpy(
    T : Any,
    a : list[float],
    symbolic : bool = False
    ) -> casadi.casadi.Function | casadi.casadi.SX:
```

Compose NASA7 specific enthalpy parameterization.

Parameters

`T : Any`

Temperature variable (symbolic or numeric).

`a : list[float]`

List of 7 NASA7 coefficients.

```
symbolic : bool = False
```

Whether to return a symbolic expression.

entropy

```
entropy(
    T : Any,
    a : list[float],
    symbolic : bool = False
    ) -> casadi.casadi.Function | casadi.casadi.SX:
```

Compose NASA7 specific entropy parameterization.

Parameters

```
T : Any
```

Temperature variable (symbolic or numeric).

```
a : list[float]
```

List of 7 NASA7 coefficients.

```
symbolic : bool = False
```

Whether to return a symbolic expression.

compose

```
compose(
    cls : Any,
    name : str,
    T : casadi.casadi.SX | numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64
    data : list[list[float]],
    symbolic : bool = False
    ) -> list[casadi.casadi.Function | casadi.casadi.SX | numpy.ndarray[tuple[typing.Any,
```

Compose a list of NASA7 functions for given data.

Parameters

```
name : str
```

Name of the function ("specific_heat", "enthalpy", or "entropy", *i.e* the static methods of this class).

```
T : casadi.casadi.SX | numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]] |
```

Temperature variable (symbolic).

```
data : list[list[float]]
```

List of NASA7 coefficient sets for each temperature range.

```
symbolic : bool = False
```

Whether to return symbolic expressions or CasADi functions. Only relevant if `T` is symbolic.

from_species

```
from_species(
    cls : Any,
    species : Species,
    T : SX
    ) -> typing.Self:
```

Create a `Nasa7Thermo` object from a Cantera species.

Parameters

```
species : Species
```

Cantera species object, with NASA7 thermodynamic data.

```
T : SX
```

Temperature variable (symbolic).

4.3 Symbolic mixture properties

5 Transport properties

aka majordome_engineering.transport

5.1 EffectiveThermalConductivity

Class `EffectiveThermalConductivity` implements static methods for the evaluation of properties; its name is quite long, let's start by getting an alias before evaluating the desired models:

```
etc = EffectiveThermalConductivity()
```

Using Maxwell approximation, we could estimate the the effective thermal conductivity of a packed bed of particles in a *matrix* of air; assume particles loosly embeded in air and the following properties; the computed effective thermal conductivity is shown to approach the air limit:

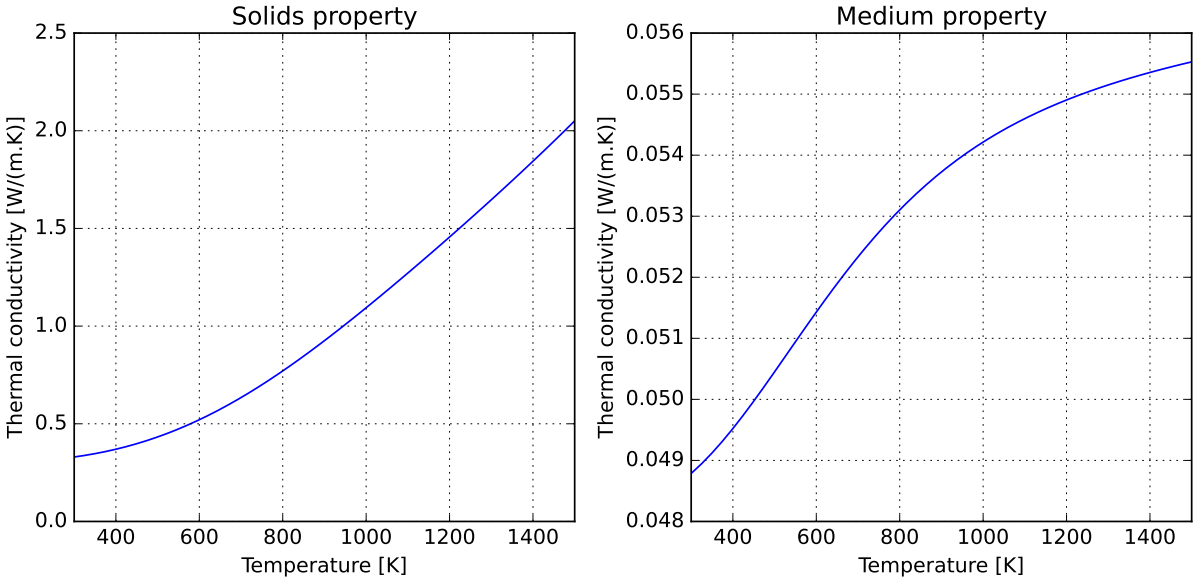
```
phi = 0.30   # Loosely packed solids [30%v]
k_g = 0.025  # Air thermal conductivity [W/(m.K)]
k_s = 1.000  # Solids thermal conductivity [W/(m.K)]

etc.maxwell_garnett(phi, k_g, k_s)
```

```
0.05396039603960396
```

For the limit of high temperatures, it is usually important to account for particle-particle radiation heat transfer; this introduces a T^3 dependence on temperature, as one should expect by linearizing Stefan-Boltzmann law.

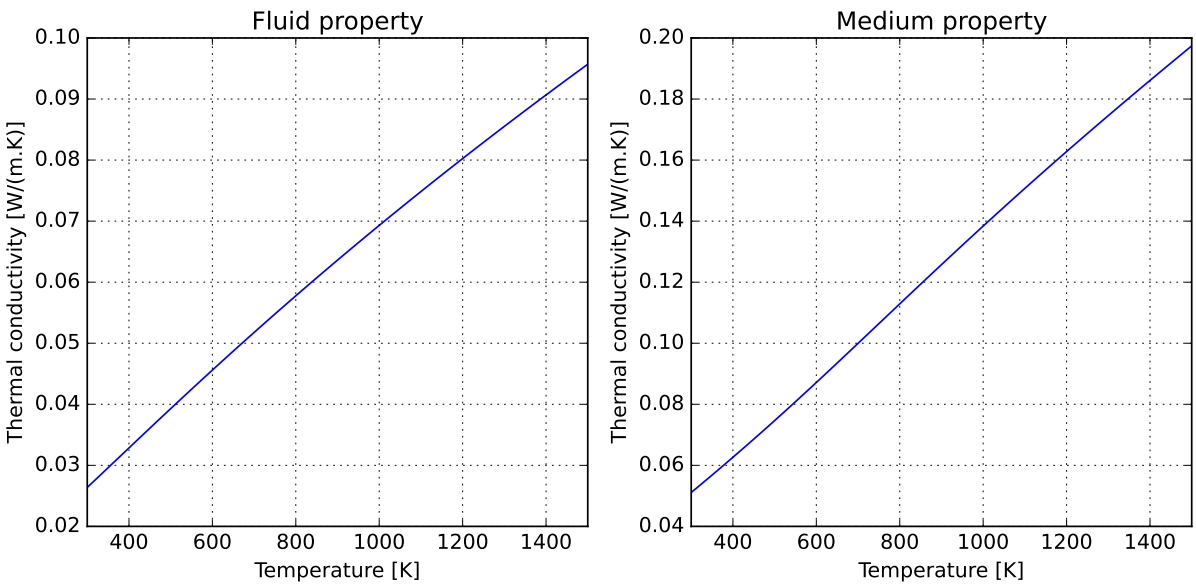
Please notice that these models compute different things; while Maxwell approximation computes the medium properties (to approximate matrix-inclusion as a single domain), Singh's model accounts only for solids properties. One might wish to combine them (warning: unverified validity!) to evaluate overall medium thermal conductivity. See the references in the class documentation for further discussion, specially the extension proposed by Kiradjiev (2019), which leads to a result similar to the assymptotic behavior displayed below.

```
d_p = 0.005
eps = 0.9

T = np.linspace(300, 1500, 50)
k_eff_s = etc.singh1994(T, phi, d_p, k_s, eps)
k_eff_m = etc.maxwell_garnett(phi, k_g, k_eff_s)

plot_etc((T, k_eff_s, k_eff_m)).resize(10, 5)
```



Due to kinetic theory implications, gas thermal conductivity tend to have a positive slope in temperature; the following illustrates how this can be accounted for and the roughly linear behaviour introduced when computing medium properties within air.

```
gas = ct.Solution("airish.yaml")
sol = ct.SolutionArray(gas, (T.shape[0],))
sol.TP = T, None

k_gas = sol.thermal_conductivity
k_eff_m = etc.maxwell_garnett(phi, k_gas, k_eff_s)

plot_etc((T, k_gas, k_eff_m)).resize(10, 5)
```



5.2 SolutionDimless

A dimensionless numbers calculator is provided for gas flows; it currently has a certain number of groups which are all evaluated by definition (which might change according to your field, please check the docs). The mechanics of using the class can be resumed to:

1. loading a mechanism file (Cantera YAML)
2. setting the state of the solution
3. evaluating the required properties
4. (optional) displaying a report

The meaning of the tuple of arguments provided to `set_state` is specified by `tuple_name="TPX"`, which defaults to temperature, pressure, and molar proportions. Any triplet allowed by Cantera can be specified here.

```
Tw = 1000.0  # Wall temperature [K]
U = 10.0     # Characteristic velocity [m/s]
D = 0.05     # Pipe diameter [m]
L = 1.0      # Pipe length [m]

calculator = SolutionDimless("airish.yaml")
calculator.set_state(300.0, 101325.0, "N2: 1", tuple_name="TPX")

Re   = calculator.reynolds(U, D)
Pr   = calculator.prandtl()
Sc   = calculator.schmidt()
print(calculator.report())
```

```
--------  ------------  ---------------
Reynolds  31460.9       U=10.0, L=0.05
Prandtl       0.709327
Schmidt       0.761752  mix_diff_coeffs
--------  ------------  ---------------
```

If you prefer to have direct access to the internal solution, you can set properties as usual in Cantera, but you need to keep in mind to call `update()` to refresh the internal state of the calculator. Every time the properties are updated, the internal buffer of computed dimensionless numbers is refreshed, as you migth notice in the following table.

```
calculator.solution.TPX = 300.0, 101325.0, "N2: 1"
calculator.update()

Pe_m = calculator.peclet_mass(U, L)
Pe_h = calculator.peclet_heat(U, L)
Gr   = calculator.grashof(Tw, D)
Ra   = calculator.rayleigh(Tw, D)
print(calculator.report())
```

```
-------------  ----------------  ------------------------------
Péclet (mass)  479308            U=10.0, L=1.0, mix_diff_coeffs
Péclet (heat)  446321            U=10.0, L=1.0
```

```
Grashof              1.13242e+07  Tw=1000.0, H=0.05, g=9.80665
Rayleigh             8.03259e+06  Tw=1000.0, H=0.05, g=9.80665
-------------  ----------------  ------------------------------
```

5.3 SutherlandFitting

To the author's knowledge, there is no standard tool to convert Cantera transport data to Sutherland parameters for use with OpenFOAM, what led to the motivation to develop `SutherlandFitting`. This simple class wraps a Cantera solution object, which it makes use for fitting Sutherland parameters to export as a table (to be used elswhere), and also allows for retrieving a converted database in OpenFOAM compatible format. The following example should be self-explanatory:

```
T = np.linspace(500, 2500, 100)

sutherland = SutherlandFitting("airish.yaml")
sutherland.fit(T, species_names=["O2", "N2"])

coef = sutherland.coefs_table
coef
```

	species	As [uPa.s]	Ts [K]	RMSE [uPa.s]
0	O2	1.873876	229.321211	0.527484
1	N2	1.619350	226.141806	0.470152

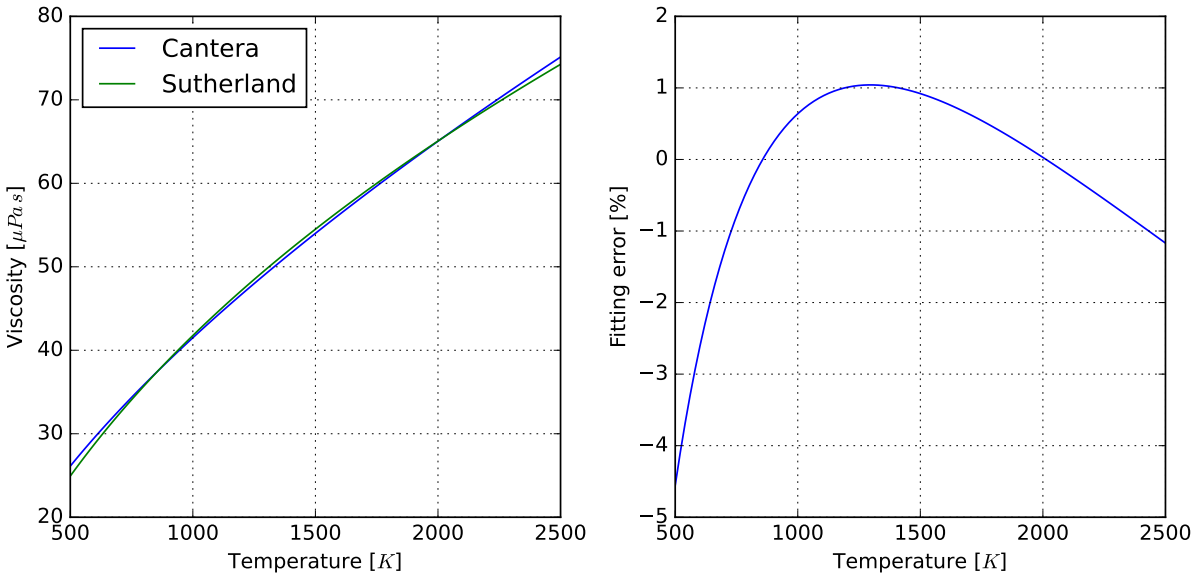
If needed, it is also possible to have direct access to the viscosity data used in parameter fitting:

```
sutherland.viscosity.head()
```

	T	O2	N2
0	500.000000	30.045456	26.123142
1	520.202020	30.886884	26.845049
2	540.404040	31.713820	27.554780
3	560.606061	32.527146	28.253075
4	580.808081	33.327662	28.940602

Because RMSE compresses all the error in a single value, you can check graphically where the deviations happen in the interval:

```
plot = sutherland.plot_species("N2")
```



Finally, it *also* responds to its initial goal of exporting values in OpenFOAM format:

```
print(sutherland.to_openfoam())
```

```
/* --- RMSE 0.5274842993902649 ---*/
"O2"
{
    transport
    {
        As  1.8738763185e-06;
        Ts  2.2932121082e+02;
    }
}
/* --- RMSE 0.4701517988286822 ---*/
"N2"
{
    transport
    {
        As  1.6193495479e-06;
        Ts  2.2614180620e+02;
    }
}
```

5.4 WSGGRadlibBordbar2020

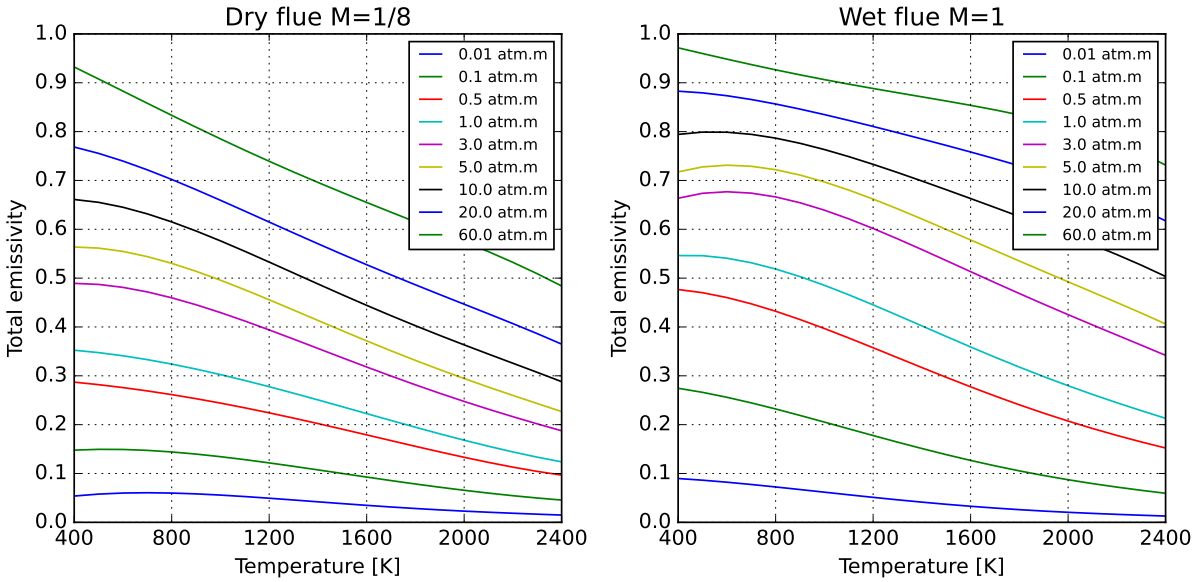
Radiative properties of gases are often required when participating media characteristics imply so. Here we implement the WSGG model of Sadeghi et al. (2021) in class `WSGGRadlibBordbar2020`. The basic usage of the model to estimate total emissivity is done as follows:

```
model = WSGGRadlibBordbar2020()
model(L=1, T=1000, P=101325, x_h2o=0.18, x_co2=0.08, fvsoot=0.0)
```

```
np.float64(0.18368577452587137)
```

5.4.1 Validation of Bordbar WSGG implementation

Evaluation of the model against original source of Bordbar (2014) is satisfactory, as follows:



Absorption coefficients for pure substances only reproduce approximately values reported by Bordbar (2020).

```
model(L=1, T=300, P=101325, x_h2o=0, x_co2=1)
model.absorption_coefs[1:]
```

```
array([3.388079e-02, 4.544269e-01, 4.680226e+00, 1.038439e+02])
```

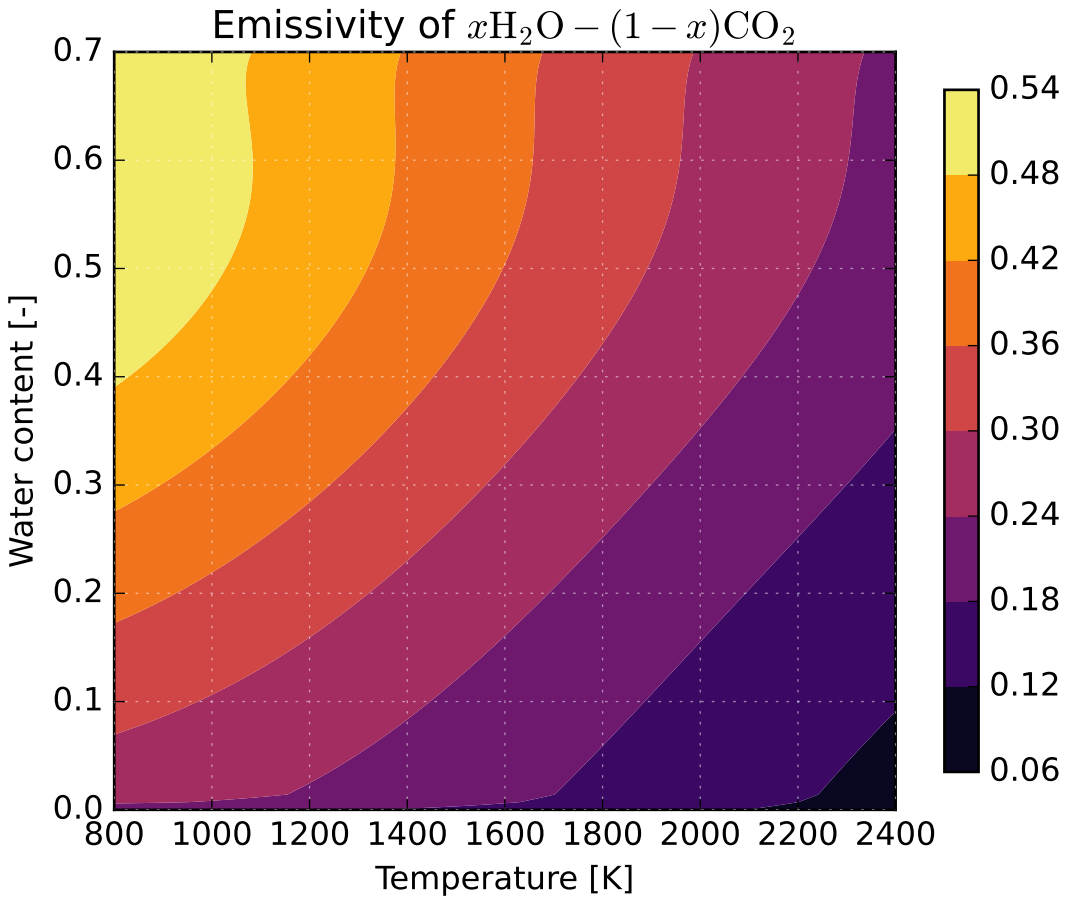
```
model(L=1, T=300, P=101325, x_h2o=1, x_co2=0)
model.absorption_coefs[1:]
```

```
array([ 0.07703541,  0.8242941 ,  6.854761  , 65.93653   ])
```

5.4.2 Application to radiative combustion

Below we illustrate the application of `WSGGRadlibBordbar2020` to predict the emissivity of $x\mathrm{H_2O} - (1-x)\mathrm{CO_2}$ mixtures over a broad temperature range applicable to the analysis of combustion processes. One observes an important participation of flue gases, especially at lower temperatures.

```
wsgg = WSGGRadlibBordbar2020()

@np.vectorize
def emissivity(T, X, L=1, P=101325):
    return wsgg(L=L, T=T, P=P, x_h2o=X, x_co2=1-X, fvsoot=0.0)

T = np.linspace(800, 2400, 100)
X = np.linspace(0, 0.7, 100)

T, X = np.meshgrid(T, X)
eps = emissivity(T, X)
plot_emissivity(T, X, eps)
```


Emissivity of $x\mathrm{H_2O} - (1-x)\mathrm{CO_2}$
Water content [-]
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
800
1000
1200
1400
1600
1800
2000
2200
2400
Temperature [K]
0.54
0.48
0.42
0.36
0.30
0.24
0.18
0.12
0.06

6 Fluent files

aka majordome_simulation.fluent

6.1 FluentFvParticlesParser

This class provides functionality for parsing particle track files. Fluent export DPM results in several formats; `majordome` supports *fieldview* particle tracks through `FluentFvParticlesParser`. Loading a file is as simple as:

You can retrieve the names of columns and number of individual tracks with the following properties:

```
(['x', 'y', 'z', 'residence_time', 'time_step'], 2)
```

For convenience, tracks can be recovered as data frames through object indexing:

	x	y	z	residence_time	time_step
0	0.128501	0.537214	-0.622358	0.000000	0.000000e+00
1	0.128493	0.537929	-0.619682	0.000107	1.697420e-07
2	0.128493	0.537964	-0.619554	0.000112	2.738057e-07
3	0.128488	0.538534	-0.617419	0.000176	4.008972e-07
4	0.128477	0.539842	-0.612481	0.000299	7.041471e-07

6.2 FluentInterpolationParser

Sometimes is is also useful to parse interpolation files. It enables one to compute new quantites for an interpolation files may be useful, especially in reasearch settings. Class `FluentInterpolationParser` is able to read the contents of an interpolation file exported in plain text format. Below you find an example of parsing.

```
            x0 in range (        11:        16)
            x1 in range (        17:        22)
      pressure in range (        23:        28)
    x-velocity in range (        29:        34)
    y-velocity in range (        35:        40)
   temperature in range (        41:        46)
             k in range (        47:        52)
         omega in range (        53:        58)
```

Names of variables are provided by property `variable_names`:

```
['x0',
 'x1',
 'pressure',
 'x-velocity',
 'y-velocity',
 'temperature',
 'k',
 'omega']
```

Valid names can be used to retrieve data for a given variable:

```
array([0.1, 0.1, 0.2, 0.3, 0.4])
```

The whole data table can be retreived with `data` property, where each variable correspond to a column:

```
array([[0.1, 1. , 9. , 9. , 9. , 9. , 9. , 9. ],
       [0.1, 2. , 1. , 1. , 1. , 1. , 1. , 1. ],
       [0.2, 3. , 2. , 2. , 2. , 2. , 2. , 2. ],
       [0.3, 4. , 3. , 3. , 3. , 3. , 3. , 3. ],
       [0.4, 5. , 4. , 4. , 4. , 4. , 4. , 4. ]])
```

7 Common

aka majordome_utilities.common

This module provides a set of utilities that are whether too general purpose for integrating another module or still waiting to find their definitive home. These include constants, type aliases, and functionalities of different kinds. Regarding this last kind of components, this tutorial aims at illustrating their use for practical documentation.

7.1 RelaxUpdate and StabilizeNvarsConvergenceCheck

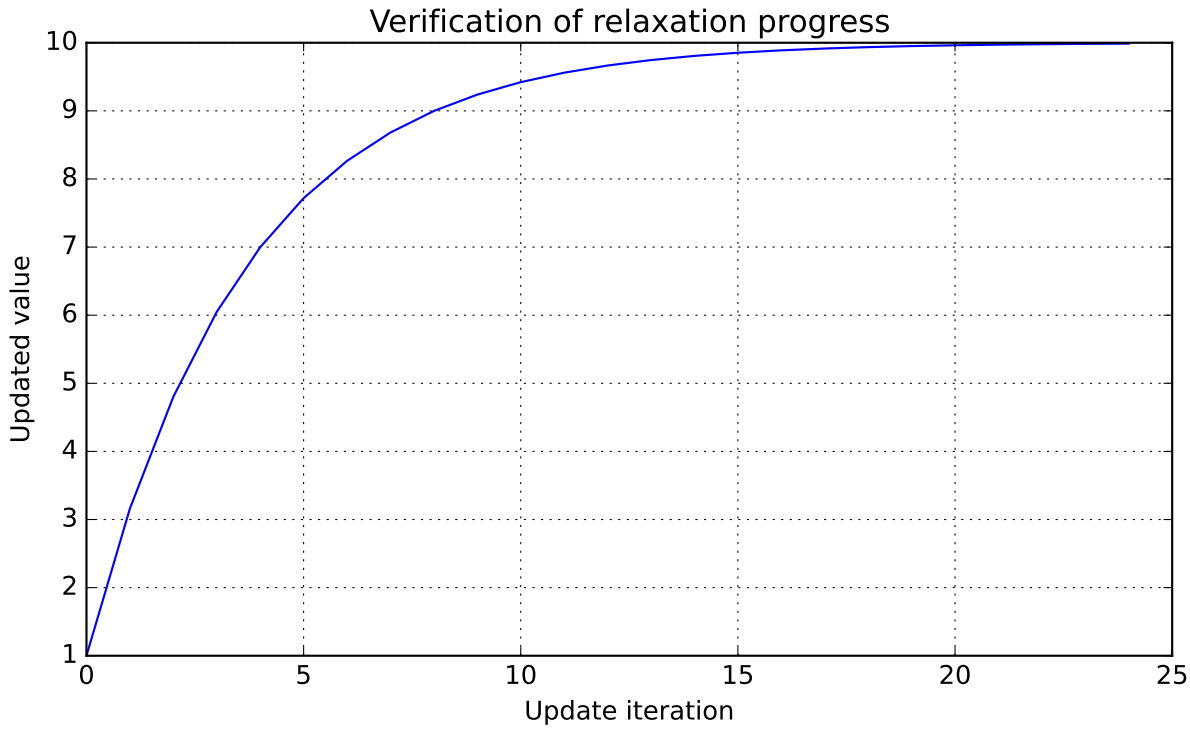
Simplest of relaxation methods; assume you have a new updated solution $A^{\star}_{new}$ for a problem whose past state was A_{old}, then the manager will ensure the following relaxation will be applied to compute the next solution state A_{new} to be used in whatever you are computing:

$$
\begin{aligned}
A_{new} &= \alpha A_{old} + (1-\alpha) A^{\star}_{new} \\
A_{old} &= A_{new}
\end{aligned}
$$

Notice that in this formulation, α (or `alpha` in the API) represents the fraction of old solution to be used in the *smearing* process. Below we illustrate the effect of a step function H valued at 10 from the begining over consecutive updates (here we do not test for convergence, as that is problem specific and for this simple case the required number of steps could be evaluated by hand, take some time to try!).

```
alpha = 0.76
niter = 50

single = np.ones(1)
relaxer = RelaxUpdate(single, alpha)

opts = dict(n_vars=1, max_iter=niter, patience=3, rtol=0.001)
converged = StabilizeNvarsConvergenceCheck(**opts)

history = np.zeros(niter+1)
history[0] = single[0]

H = np.asarray([10])

for n in range(niter):
    single[:] = relaxer(H)
    history[n+1] = single[0]

    if converged(single[0]):
        history = history[:n+2]
        break
```

```
_ = plot_history(history)
```



8 Documents

aka majordome_utilities.documents

SignatureEntry

```
SignatureEntry(
    fn : type,
    *,
    greedy : bool = True
    ) -> None:
```

Builds the representation of a function signature.

Parameters

`fn : type`

The function to build the signature for.

`greedy : bool = True`

Whether to raise errors for missing docstrings or parameters.

documentation

```
documentation(
    self : Any,
    **kwargs : Any
    ) -> None:
```

Generates the markdown documentation for the function.

Parameters

`title : str = None`

The title to use for the section header and callout. By default, the function name is used.

8.1 Tutorial

```python
class Dummy:
    """ A dummy class for testing purposes. """
    def method(self, a, b):
        """ This is a method.

        Parameters
        ----------
        a : int
            This is the first parameter.
        b : int
            This is the second parameter.
        """
        pass

    @staticmethod
    def static_method(a, b):
        """ This is a static method.

        Parameters
        ----------
        a : int
            This is the first parameter.
        b : int
            This is the second parameter.
        """
        pass

    @classmethod
    def class_method(cls, a, b):
        """ This is a class method.

        Parameters
        ----------
        a : int
            This is the first parameter.
        b : int
            This is the second parameter.
        """
        pass

    @property
    def prop(self) -> int:
        """ This is a property. """
        return 42
```

Documentation of all members at once is discouraged, but it is possible to do so by calling all methods of ‘`DocumentedClass`:

```python
doc = DocumentedClass(Dummy)
doc.constructor()
```

```
doc.members()
doc.properties()
doc.classmethods()
doc.staticmethods()
```

For generating more organized documentation, it is possible to selectively include or exclude static methods, class methods and properties. This allows to generate separate sections for each type of member, which can be more readable and easier to navigate.

The constructor:

Dummy

```
Dummy():
```

A dummy class for testing purposes.

Only normal methods:

method

```
method(
    self : Any,
    a : Any,
    b : Any
    ):
```

This is a method.

Parameters

`a : Any`

This is the first parameter.

`b : Any`

This is the second parameter.

Only static methods:

static_method

```
static_method(
    a : Any,
    b : Any
    ):
```

This is a static method.

Parameters

`a : Any`

This is the first parameter.

`b : Any`

This is the second parameter.

Only class methods:

class_method

```
class_method(
    cls : Any,
    a : Any,
    b : Any
    ):
```

This is a class method.

Parameters

`a : Any`

This is the first parameter.

`b : Any`

This is the second parameter.

Only properties:

prop

```
prop(self : Any): -> int:
```

This is a property.

8.2 Examples

A minimal function with no parameters and no docstring:

```python
def awesome_useless():
    pass
```

awesome_useless

```
awesome_useless():
```

(missing summary)

A function with variadic parameters and no documented parameters:

```python
def awesome_useless(*args, **kwargs):
    """ Some useless function. """
    pass
```

awesome_useless

```
awesome_useless(
    *args : Any,
    **kwargs : Any
    ):
```

Some useless function.

Parameters

```
*args : Any
```

(missing description)

A function with variadic parameters with type annotation and explictly documented keyword parameter:

```python
def awesome_useless(*args: tuple[int, ...], **kwargs):
    """ Some useless function.

    Parameters
    ----------
    args : tuple
        Positional arguments. This is intended to be a catch-all for
        any number of positional arguments. Let's write a long text.
    some_kwarg : int = 1
        Keyword argument. This is intended to be a catch-all for any
        number of keyword arguments. Let's write a long text.
    """
    pass
```

awesome_useless

```
awesome_useless(
    *args : tuple[int, ...],
    **kwargs : Any
    ):
```

Some useless function.

Parameters

`*args : tuple[int, ...]`

Positional arguments. This is intended to be a catch-all for any number of positional arguments. Let's write a long text.

`some_kwarg : int  = '1'`

Keyword argument. This is intended to be a catch-all for any number of keyword arguments. Let's write a long text.

A function with a positional-only parameter, a keyword-only parameter and variadic parameters:

```
def awesome_useless(a: int, /, b, **kwargs):
    """ Some useless function. """
    pass
```

awesome_useless

```
awesome_useless(
    a : int,
    /,
    b : Any,
    **kwargs : Any
    ):
```

Some useless function.

Parameters

`a : int`

(missing description)

`b : Any`

(missing description)

Function with all kinds of parameters:

```
def awesome_useless(x, /, y: int = 0, *, z: int = 0, **kwargs) -> None:
    """ Some useless function.

    Parameters
    ----------
    x : int
        Nothing to say about this.
    y : int
        Nothing to say about this.
    z : int
        Nothing to say about this.
    xx : int
        Nothing to say about this.
    """
    pass
```

awesome_useless

```
awesome_useless(
    x : Any,
    /,
    y : int = 0,
    *,
    z : int = 0,
    **kwargs : Any
    ) -> None:
```

Some useless function.

Parameters

`x : Any`

Nothing to say about this.

`y : int = 0`

Nothing to say about this.

`z : int = 0`

Nothing to say about this.

`xx : int = None`

Nothing to say about this.

9 Argument parsing

aka majordome_utilities.parsing

9.1 FuncArguments

```
def f(*args, **kwargs):
    """ Arbitrary interface function for illustration. """
    f.parser.update(*args, **kwargs)

    try:
        print(f"a = {f.parser.get('a')}, b = {f.parser.get('b')}")
        f.parser.close()
    except Exception as err:
        print(err)
        pass
```

- Arguments are both positional-mandatory

```
f.parser = FuncArguments()
f.parser.add("a", index=0)
f.parser.add("b", index=1)

f(3)
f(3, 4)
```

```
Cannot retrieve mandatory positional argument at position 1
a = 3, b = 4
```

- Arguments are both keyword-only

```
f.parser = FuncArguments()
f.parser.add("a", default="foo")
f.parser.add("b", default="bar")

f(3)
f(3, 4)
f(3, b=4)
f(a=3, b=4)
```

```
a = foo, b = bar
Too many positional arguments, got 1 but only 0 were used.
a = foo, b = bar
Too many positional arguments, got 2 but only 0 were used.
a = foo, b = 4
```

```
Too many positional arguments, got 1 but only 0 were used.
a = 3, b = 4
```

- One positional and another (maybe) positional

```
f.parser = FuncArguments()
f.parser.add("a", index=0)
f.parser.add("b", index=1, default=6)

f(3)
f(3, 4)
f(3, 4, b=4)
```

```
a = 3, b = 6
a = 3, b = 4
Cannot have both positional and keyword version of b (1) simultaneously
```

- Badly configured

```
f.parser = FuncArguments()

try:
    f.parser.add("a")
    f.parser.add("b")
except Exception as err:
    print(err)
```

```
Argument must be either positional or keyword, cannot be neither: a
```

9.2 Creating class constructors

You should probably not do this, but it is quite fun!

```
def _init_some_class(cls):
    """ Decorator to enhance SomeClass with argument parsing. """
    orig_init = cls.__init__

    # Maybe stash for later use?
    # cls.__orig_init__ = orig_init

    parser = FuncArguments(greedy_args=False, pop_kw=True)
    parser.add("a", 0)
    parser.add("x", default=None)

    @wraps(orig_init)
    def new_init(self, *args, **kwargs):
        parser.update(*args, **kwargs)
        a = parser.get("a")
        x = parser.get("x")

        orig_init(self, *parser.args, **parser.kwargs)
```

```
        parser.close()

        # -- logic goes here
        print(f"some a = {a}")
        print(f"some x = {x}")

        return None

    cls.__init__ = update_wrapper(new_init, orig_init)
    return cls

class BaseClass:
    def __init__(self, *args, **kwargs) -> None:
        print(f"args   = {args}")
        print(f"kwargs = {kwargs}")

@_init_some_class
class SomeClass(BaseClass):
    def __init__(self, *args, **kwargs) -> None:
        super().__init__(*args, **kwargs)

some = SomeClass(1, 2, 3, x=1, y=2)
```

```
args   = (1, 2, 3)
kwargs = {'y': 2}
some a = 1
some x = 1
```

10 PDF Tools

aka majordome_utilities.pdftools

10.1 PdfToTextConverter

Below we illustrate the usage of `PdfToTextConverter`. Please notice that data curation of extracted texts is still required if readability is a requirement. If quality of automated extractions is often poor for a specific language, you might want to search the web how to *train tesseract*, that topic is not covered here.

Note: this note assumes tesseract and poppler, and ImageMagick are available in system path. Under Windows you might struggle to get them all working together, please check Majordome's Kompanion for automatic installation.

Install dependencies on Ubuntu 22.04:

```
sudo apt install tesseract-ocr imagemagick poppler-utils
```

In case of Rocky Linux 9:

```
sudo dnf install tesseract tesseract-langpack-eng ImageMagick poppler-utils
```

Assuming the dependencies are found in the path, it is simply a matter of creating a converter:

```
converter = PdfToTextConverter()
```

For generated PDF (not scanned documents), it is much faster to avoir using OCR; below we show the metadata from a paper:

```
data = converter("data/docs/paper.pdf", use_ocr=False)
data.meta
```

```
{'/Author': "W. Dal'Maz Silva",
 '/CreationDate': "D:20170403224009+05'30'",
 '/Creator': 'Elsevier',
 '/CrossMarkDomains[1]': 'elsevier.com',
 '/CrossMarkDomains[2]': 'sciencedirect.com',
 '/CrossmarkDomainExclusive': 'true',
 '/CrossmarkMajorVersionDate': '2010-04-23',
 '/ElsevierWebPDFSpecifications': '6.5',
 '/Keywords': 'Hardness measurement; Martensite; Low-alloy steel; Precipitation',
 '/ModDate': "D:20170403224009+05'30'",
 '/Subject': 'Materials Science & Engineering A, 693 (2017) 225-232. doi:10.1016/j.msea.2017
 '/Title': 'Carbonitriding of low alloy steels_ Mechanical and metallurgical responses',
 '/doi': '10.1016/j.msea.2017.03.077',
 '/robots': 'noindex'}
```

For scanned documents, by default if OCR is not enabled it will be used as a fallback method for text extraction:

```
data = converter("data/docs/scanned.pdf", last_page=1)
data.content[:500]
```

```
'549\n\n5.. Uber die von der molekularkinetischen Theorie\nder Wirme geforderte Bewegung vor
\nkinetischen Theorie der Warme in Flissigkeiten suspendierte\nKorper von mikroskopisch sich
\nkularbewegung der Wirme Bewegungen von solcher GrifSe\nausfiihren miissen, daB diese Beweg
```

11 Plotting

aka majordome_utilities.plotting

11.1 MajordomePlot

One might ask why a wrapper around Matplotlib's `Figure` and `Axes` is necessary, and I would strongly agree that it isn't. However, keeping all the plots in a report consistent is a huge effort, and being able to standardize the plots without having to write new themes is a nice feature. That's the main goal of `MajordomePlot`: simplifying the plotting of publication-quality figures across Majordome (and beyond!). Below we illustrate how to use this functionality to ensure all the plots in a project look the same.

MajordomePlot

```
MajordomePlot(
    shape : tuple[int, int] = (1, 1),
    style : str = 'classic',
    *,
    size : tuple[float, float] | None = None,
    xlabel : str | list[str] | None = None,
    ylabel : str | list[str] | None = None,
    opts : dict[str, typing.Any] = {'facecolor': 'white'}
    ) -> None:
```

Handles the creation and management of plots.
Provides a handle for creating and managing plots using Matplotlib. It is aimed at standardizing the plot creation process across the Majordome framework.

Parameters

`shape : tuple[int, int] = (1, 1)`

Shape of the plot as (n_rows, n_cols).

`style : str = 'classic'`

Matplotlib style to use for the plot.

`size : tuple[float, float] | None = None`

Size of the plot in inches as (width, height). If None, the default size will be used.

```
xlabel : str | list[str] | None = None
```

Label(s) for the x-axis. If a list is provided, each subplot will get its own label.

```
ylabel : str | list[str] | None = None
```

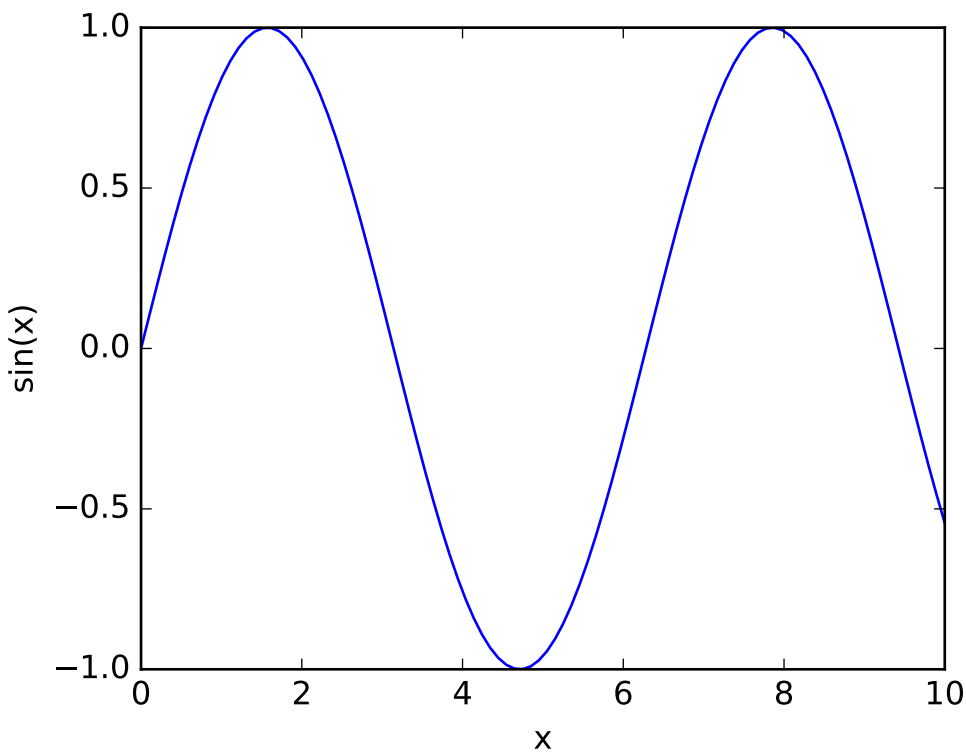
Label(s) for the y-axis. If a list is provided, each subplot will get its own label.

```
opts : dict[str, typing.Any] = {'facecolor': 'white'}
```

Additional options to pass to `plt.subplots()`.

Let's see how one would expect to use it from the above constructor signature. This is in fact almost the same as calling `plt.subplots()`, and that is what the constructor does under the hood. This is a *negative* example actually, the author has *never* used the constructor directly, but instead uses the `new` classmethod, which is a much more convenient way to create a plot. More on that later.

```
x = np.linspace(0, 10, 100)
mp = MajordomePlot(shape=(1, 1), size=(5, 4))
mp.axes[0].plot(x, np.sin(x))
mp.axes[0].set_xlabel("x")
mp.axes[0].set_ylabel("sin(x)")
pass
```



The story behind this class dates a few years back, when I was learning about Python decorators. As an exercise, I coded what later became the `new` classmethod, which is a decorator that creates a new `MajordomePlot` and passes it to the decorated function. This is a very convenient way to create

plots, as it allows you to focus on the plotting code and not worry about the boilerplate of creating the plot and setting the labels.

new

```
new(
    cls : Any,
    _func : typing.Optional[typing.Callable[~Params, NoneType]] = None,
    *,
    shape : tuple[int, int] = (1, 1),
    style : str = 'classic',
    sharex : bool = True,
    facecolor : str = 'white',
    grid : bool = True,
    size : tuple[float, float] | None = None,
    xlabel : str | list[str] | None = None,
    ylabel : str | list[str] | None = None
    ) -> typing.Callable[[typing.Callable[~Params, NoneType]], typing.Any]:
```

Wraps a function for ensuring a standardized plot.

Parameters

`_func : typing.Optional[typing.Callable[~Params, NoneType]] = None`

Function to wrap, if any.

`shape : tuple[int, int] = (1, 1)`

Shape of the plot as (n_rows, n_cols).

`style : str = 'classic'`

Matplotlib style to use for the plot. By default "classic".

`sharex : bool = True`

Whether to share x-axis among subplots.

`facecolor : str = 'white'`

Face color of the figure.

`grid : bool = True`

Whether to display grid lines.

`size : tuple[float, float] | None = None`

Size of the plot in inches as (width, height). If None, the default size will be used. By default None.

`xlabel : str | list[str] | None = None`

Label(s) for the x-axis. If a list is provided, each subplot will get its own label. By default None.

`ylabel : str | list[str] | None = None`

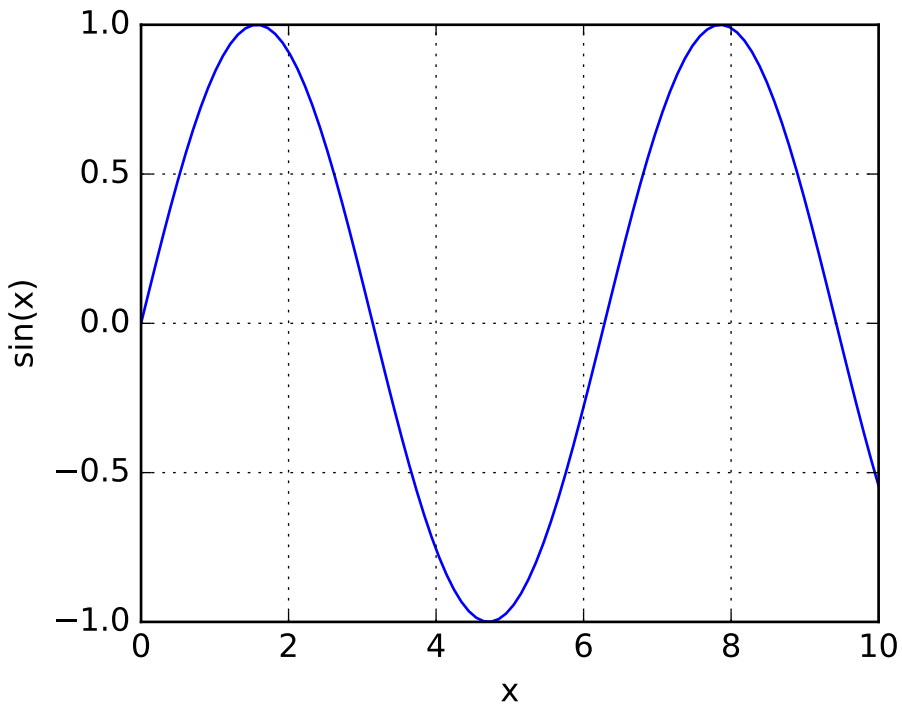
Label(s) for the y-axis. If a list is provided, each subplot will get its own label. By default None.

In the following example, we make use of this decorator to parametrize the labels of the plot. Internally, we must expect a *keyword-only* argument `plot`, which is the `MajordomePlot` instance created by the decorator, from which we can access the `Figure` and `Axes` objects as usual. The rest of the code is just standard Matplotlib code, with the exeption that the axes is raveled, so we can access it with `ax[0]` instead of `ax[0, 0]`. Returning `plot` is optional (it is done by the decorator already), but it avoids linting errors if you want to annotate the return type as `MajordomePlot`.

Here we create a function that will be used below in the examples. In summary, to get it working, you need to:

- Use decorator `@MajordomePlot.new` with all its configurable attributes; for details, please refer to its API documentation.
- Have a function with signature `func(*args, plot=None, **kwargs) -> None`, where it is recommended (for linter) to provide explictly keyword `plot=None`.
- Unpack `fig, ax = plot.subplots()` or just `_, ax = plot.subplots()`, as needed, inside the figure; these contain standard `matplotlib` figure and axes.

```
@MajordomePlot.new(size=(5, 4), xlabel="x", ylabel="sin(x)")
def plot_sin(x, *, plot, **kwargs) -> MajordomePlot:
    _, ax = plot.subplots()
    ax[0].plot(x, np.sin(x))
    return plot

plot = plot_sin(np.linspace(0, 10, 100))
```



The following elements document the members and properties of the class.

resize

```
resize(
    self : Any,
    w : float,
    h : float
    ) -> None:
```

Resize a plot with width and height in inches.

Parameters

`w : float`

Width of the plot in inches.

`h : float`

Height of the plot in inches.

savefig

```
savefig(
    self : Any,
```

```
    filename : str | pathlib._local.Path,
    **kwargs : Any
    ) -> None:
```

Wrapper for saving a figure to file.

Parameters

`filename : str | pathlib._local.Path`

Path to save the figure to.

`kwargs : None = None`

Additional keyword arguments to pass to `Figure.savefig()`.

subplots

```
subplots(self : Any): -> tuple[matplotlib.figure.Figure, list[matplotlib.axes._axes.Axes]]
```

Provides access to underlying figure and axes.

figure

```
figure(self : Any): -> Figure:
```

Provides access to underlying figure.

axes

```
axes(self : Any): -> list[matplotlib.axes._axes.Axes]:
```

Provides access to underlying axes.

11.2 PowerFormatter

PowerFormatter

```
PowerFormatter(**kwargs : Any): -> None:
```

Formatter for power of ten in numerical axes.

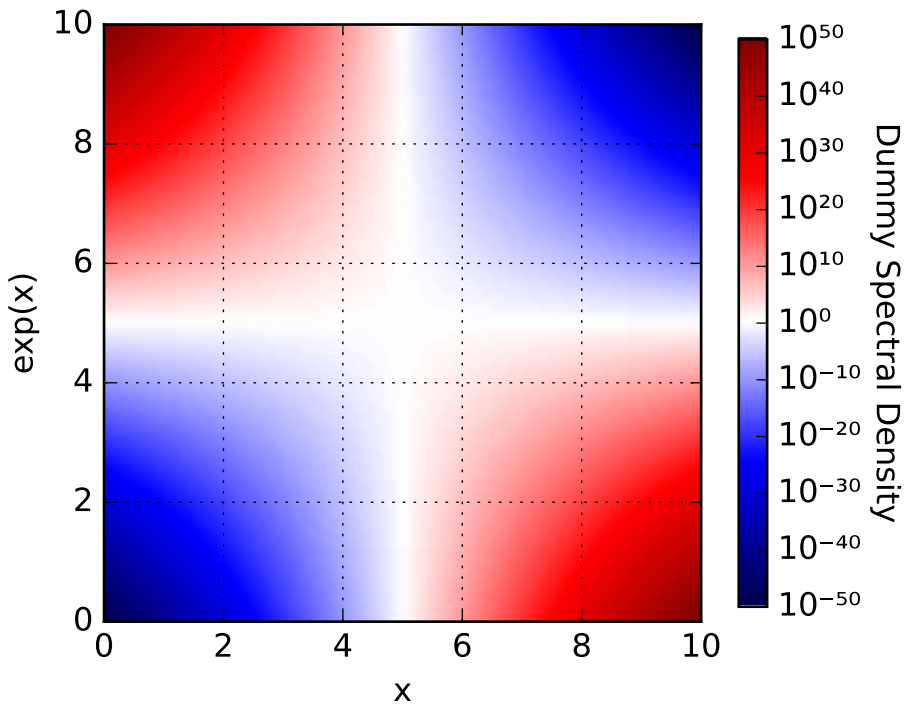
Parameters

`values : str = None`

String of characters to be replaced by their superscript counterparts. By default "0123456789-".

```
supers : str = None
```

String of superscript characters corresponding to `values`. By default " ¹²³ ".



11.3 General utilities

centered_colormap

```
centered_colormap(
    name : str,
    vmin : float,
    vmax : float,
    vcenter : float = 0.0
    ) -> LookupTable:
```

Ensure the center of a colormap is at zero.

Parameters

`name : str`

Name of the colormap to use.

`vmin : float`

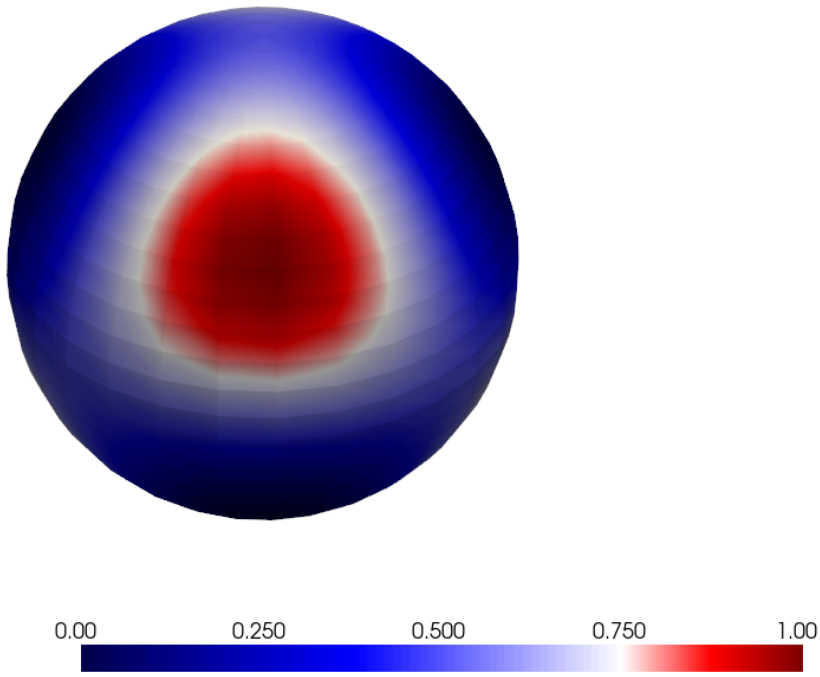
Minimum value of the data range.

`vmax : float`

Maximum value of the data range.

`vcenter : float = 0.0`

Center value of the colormap, by default 0.



12 Developer Guide

12.1 Prerequisites

The environment for developing Majordome requires at least a reasonably mondern version of Python and Rust. Documentation depends on Quarto. The exact versions may change over time, but as of the last update, the following (minimum) versions are recommended:

- Python 3.12
- Rust 1.90
- Quarto 1.8

The development focuses on portability and ease of setup, so by default any dependency relying on a C/C++ compiler is forbidden. This means that the development environment can be set up on any platform without needing to install additional compilers or tools.

12.2 Command reference

12.2.1 Windows PowerShell

Windows development is performed using the `build.ps1` script, you find it in the project root. It is fully documented, but for convenience, here is a quick reference to the most common commands.

Building the package:

```
# Install the Python package in development mode:
./build.ps1 -FromPip

# Install the Python package with release optimizations and create wheel:
./build.ps1 -FromPip -PackageDist -FlagRelease
```

Testing the package:

```
# Check Rust code with backtraces enabled.
./build.ps1 -RustCheck -FlagBacktrace

# Run Python tests.
./build.ps1 -TestRust

# Run Python tests.
./build.ps1 -TestPython
```

Documentation builds:

```
# Only HTML documentation:
./build.ps1 -PackageDocs

# Also generate PDF documentation:
./build.ps1 -PackageDocs -DocsPdf

# Erase everything and do not use any cached files:
./build.ps1 -PackageDocs -FreshDocs

# XXX Must be used alone, as build might fail.
# Publish the documentation to GitHub Pages:
./build.ps1 -PublishDocs
```

12.2.2 Unix-like shell

For developing the package, consider doing so from within a virtual environment; that will makes things easier to check versions and pin them properly in package dependencies, as needed. Following the next snippet, install, activate, upgrade pip, install build tools, and install the package in editable mode with all optional dependencies:

```
# Create a virtual environment
python -m venv venv

# Activate the virtual environment (Windows PowerShell)
. venv/Scripts/Activate.ps1

# Activate the virtual environment (Unix-like)
. venv/bin/activate

# Upgrade pip
python -m pip install --upgrade pip

# Install build tools
python -m pip install --upgrade --force-reinstall build wheel
python -m pip install --upgrade --force-reinstall setuptools
python -m pip install --upgrade --force-reinstall setuptools_rust

# Install the package in editable mode with all optional dependencies
pip install -e .[full]
```

12.3 Documentation

Majordome code is documented using Numpydoc style docstrings, and the documentation is generated by module `majordome_utilities.documents` using Quarto markdown. This design choice reflects the academic aim of the packaged, with reporting and notebook support as a first-class citizen. Differently from usual packages, the `.qmd` files live along the modules they document, except for the main documentation file `developers.qmd`, which is located in the `_docs` folder. This allows to keep the documentation close to the code, and to easily update it when the code changes.

12.4 Distribution

Note for now the distribution of binaries is being done only through GitHub. Once a stable version is reached, consider updating this section for PyPI.

For distribution, maintainers need to do the following:

```
python -m pip install --upgrade twine
python -m pip install --upgrade build
python -m build
python -m twine upload --repository majordome dist/*
```

12.5 Dependency management

Generate the list of `full` dependencies using the following snipped; make sure to sanitize the outputs and update the corresponding section in `pyproject.toml`. There might be some duplicate dependencies that **shall** agree between themselves.

```
from textwrap import indent
import tomllib


def get_full_deps(data):
    deps = data["project"]["optional-dependencies"]
    deps = [f'"{p}"' for pkgs in deps.values() for p in pkgs]
    deps = indent(",\n".join(sorted(set(deps))), "    ")
    return f"full = [\n{deps}\n]"


def gen_full_deps(project="pyproject.toml", output="sandbox-deps.toml"):
    with open(project, "rb") as fr:
        with open(output, "w") as fp:
            fp.write(get_full_deps(tomllib.load(fr)))


gen_full_deps()
```

12.6 Todo lists

12.6.1 Quick needs

- Automate documentation builds with GitHub actions.
- Update and simplify containerized builds for Linux.

12.6.2 Wishlist

- Implement diffusional Grashof number in `majordome_engineering.transport.SutherlandFitting`, which is related to mass transfer arising because of the buoyant force caused by the concentration inhomogeneities.
- Create a data converter to create Ansys Fluent SCM files directly from Cantera YAML (species data and reactions).
- Add generation of TUI commands for Fluent expression generator (and further expand to general Fluent setup).
- Migrate `gpxpy` usage to plain XML for better maintainability.

13 References

Sadeghi, Hosein, Simo Hostikka, Guilherme Crivelli Fraga, and Hadi Bordbar. 2021. "Weighted-Sum-of-Gray-Gases Models for Non-Gray Thermal Radiation of Hydrocarbon Fuel Vapors, CH4, CO and Soot." *Fire Safety Journal* 125 (October): 103420. https://doi.org/10.1016/j.firesaf.2021.103420.